

AT82.02 – Data Modeling and Management

Lab 6: MongoDB CRUD (READ) – Essential Assessment

Dataset: sample_restaurants (MongoDB Atlas Sample Dataset)

Objective

This lab evaluates students' understanding of **MongoDB READ operations** using the restaurants collection. Students must demonstrate competence in: - Basic queries and projections - Filtering and comparison operators - Logical operators - Pagination and sorting - Pattern matching (regex) - Array and embedded document queries

Task 1: Load Sample Dataset

1. Log in to MongoDB Atlas.
2. Go to **Clusters** → **Load Sample Dataset**.
3. Select **sample_restaurants** and wait until loading completes.

Task 2: MongoDB Queries

Write the MongoDB commands for each question below and **capture screenshots of both the query and the output**.

A. Basic Read & Projection

1. Display all documents in the restaurants collection.
db.restaurants.find()

```
> db.restaurants.find()
< {
  _id: ObjectId('5eb3d668b31de5d588f4292a'),
  address: {
    building: '2780',
    coord: [
      -73.98241999999999,
      40.579505
    ],
    street: 'Stillwell Avenue',
    zipcode: '11224'
  },
  borough: 'Brooklyn',
  cuisine: 'American',
  grades: [
    {
      date: 2014-06-10T00:00:00.000Z,
      grade: 'A',
      score: 5
    },
    {
      date: 2013-06-05T00:00:00.000Z,
      grade: 'A',
      score: 7
    },
    {
      date: 2012-04-13T00:00:00.000Z,
      grade: 'A',
      score: 12
    },
    {
      date: 2011-10-12T00:00:00.000Z,
      grade: 'A',
      score: 12
    }
  ],
  name: 'Riviera Caterer',
  restaurant_id: '40356018'
```

2. Display restaurant_id, name, borough, and cuisine for all restaurants.

db.restaurants.find({}, {restaurant_id:1, name:1, borough:1, cuisine:1})

```
> db.restaurants.find({}, {restaurant_id:1, name:1, borough:1, cuisine:1})
< {
  _id: ObjectId('5eb3d668b31de5d588f4292a'),
  borough: 'Brooklyn',
  cuisine: 'American',
  name: 'Riviera Caterer',
  restaurant_id: '40356018'
}
{
  _id: ObjectId('5eb3d668b31de5d588f4292b'),
  borough: 'Brooklyn',
  cuisine: 'Delicatessen',
  name: "Wilken'S Fine Food",
  restaurant_id: '40356483'
}
{
  _id: ObjectId('5eb3d668b31de5d588f4292c'),
  borough: 'Staten Island',
  cuisine: 'Jewish/Kosher',
  name: 'Kosher Island',
  restaurant_id: '40356442'
}
{
  _id: ObjectId('5eb3d668b31de5d588f4292d'),
  borough: 'Brooklyn',
  cuisine: 'Hamburgers',
  name: "Wendy'S",
  restaurant_id: '30112340'
```

3. Display restaurant_id, name, borough, and cuisine, excluding the _id field.

db.restaurants.find({}, {_id:0, restaurant_id:1, name:1, borough:1, cuisine:1})

```
db.restaurants.find({}, {_id:0,restaurant_id:1,name:1,borough:1,cuisine:1})
{
  borough: 'Brooklyn',
  cuisine: 'American',
  name: 'Riviera Caterer',
  restaurant_id: '40356018'
}
{
  borough: 'Brooklyn',
  cuisine: 'Delicatessen',
  name: "Wilken'S Fine Food",
  restaurant_id: '40356483'
}
{
  borough: 'Staten Island',
  cuisine: 'Jewish/Kosher',
  name: 'Kosher Island',
  restaurant_id: '40356442'
}
{
  borough: 'Brooklyn',
  cuisine: 'Hamburgers',
  name: "Wendy'S",
  restaurant_id: '30112340'
}
```

B. Filtering with Conditions

4. Find all restaurants located in the borough **Bronx**.

db.restaurants.find({borough:"Bronx"})

```
db.restaurants.find({borough:"Bronx"})
{
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  address: {
    building: '1007',
    coord: [
      -73.856077,
      40.848447
    ],
    street: 'Morris Park Ave',
    zipcode: '10462'
  },
  borough: 'Bronx',
  cuisine: 'Bakery',
  grades: [
    {
      date: 2014-03-03T00:00:00.000Z,
      grade: 'A',
      score: 2
    },
    {
      date: 2013-09-11T00:00:00.000Z,
      grade: 'A',
      score: 6
    },
    {
      date: 2013-01-24T00:00:00.000Z,
      grade: 'A',
      score: 10
    },
  ],
}
```

5. Find restaurants that achieved a **score greater than 90**.
db.restaurants.find({"grades.score":{\$gt:90}})

```
> db.restaurants.find({"grades.score":{$gt:90}})
```

```
< {  
  _id: ObjectId('5eb3d668b31de5d588f42a92'),  
  address: {  
    building: '65',  
    coord: [  
      -73.9782725,  
      40.7624022  
    ],  
    street: 'West 54 Street',  
    zipcode: '10019'  
  },  
  borough: 'Manhattan',  
  cuisine: 'American',  
  grades: [  
    {  
      date: 2014-08-22T00:00:00.000Z,  
      grade: 'A',  
      score: 11  
    },  
    {  
      date: 2014-03-28T00:00:00.000Z,  
      grade: 'C',  
      score: 131  
    },  
    {  
      date: 2013-09-25T00:00:00.000Z,  
      grade: 'A',  
      score: 11  
    },  
    {  
      date: 2013-04-08T00:00:00.000Z,  
      grade: 'B',  
      score: 25  
    }  
  ]  
}
```

6. Find restaurants that achieved a **score between 80 and 100**.
db.restaurants.find({"grades.score":{"\$gt:80,\$lt:90}})

```
> db.restaurants.find({"grades.score":{$gt:80,$lt:90}})
```

```
< {  
  _id: ObjectId('5eb3d668b31de5d588f42a92'),  
  address: {  
    building: '65',  
    coord: [  
      -73.9782725,  
      40.7624022  
    ],  
    street: 'West 54 Street',  
    zipcode: '10019'  
  },  
  borough: 'Manhattan',  
  cuisine: 'American',  
  grades: [  
    {  
      date: 2014-08-22T00:00:00.000Z,  
      grade: 'A',  
      score: 11  
    },  
    {  
      date: 2014-03-28T00:00:00.000Z,  
      grade: 'C',  
      score: 131  
    },  
    {  
      date: 2013-09-25T00:00:00.000Z,  
      grade: 'A',  
      score: 11  
    },  
    {  
      date: 2013-04-08T00:00:00.000Z,  
      grade: 'B',  

```


C. Pagination (Limit & Skip)

7. Display the **first 5 restaurants** in the borough Bronx.

```
db.restaurants.find({borough:"Bronx"}).limit(5)
```

```
> db.restaurants.find({borough:"Bronx"}).limit(5)
< {
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  address: {
    building: '1007',
    coord: [
      -73.856077,
      40.848447
    ],
    street: 'Morris Park Ave',
    zipcode: '10462'
  },
  borough: 'Bronx',
  cuisine: 'Bakery',
  grades: [
    {
      date: 2014-03-03T00:00:00.000Z,
      grade: 'A',
      score: 2
    },
    {
      date: 2013-09-11T00:00:00.000Z,
      grade: 'A',
      score: 6
    },
    {
      date: 2013-01-24T00:00:00.000Z,
      grade: 'A',
      score: 10
    },
    {
```

8. Display the **next 5 restaurants** in the borough Bronx after skipping the first 5.

```
db.restaurants.find({borough:"Bronx"}).skip(5).limit(5)
```

```
> db.restaurants.find({borough:"Bronx"}).skip(5).limit(5)
< {
  _id: ObjectId('5eb3d668b31de5d588f42961'),
  address: {
    building: '658',
    coord: [
      -73.81363999999999,
      40.829411000000001
    ],
    street: 'Clarence Ave',
    zipcode: '10465'
  },
  borough: 'Bronx',
  cuisine: 'American',
  grades: [
    {
      date: 2014-06-21T00:00:00.000Z,
      grade: 'A',
      score: 5
    },
    {
      date: 2012-07-11T00:00:00.000Z,
      grade: 'A',
      score: 10
    }
  ],
  name: 'Manhem Club',
  restaurant_id: '40364363'
}
{
  _id: ObjectId('5eb3d668b31de5d588f4297e'),
  address: {
```

D. Logical Operators

9. Find restaurants that do **not** prepare **American** cuisine, have a **score greater than 70**, and are located at a longitude less than -65.754168 (do **not** use \$and).

```
db.restaurants.find({  
  cuisine: { $ne: "American" },  
  "grades.score": { $gt: 70 },  
  "address.coord.0": { $lt: -65.754168 }  
})
```

```

> db.restaurants.find({
  cuisine: { $ne: "American" },
  "grades.score": { $gt: 70 },
  "address.coord.0": { $lt: -65.754168 }
})
< {
  _id: ObjectId('5eb3d668b31de5d588f42b26'),
  address: {
    building: '345',
    coord: [
      -73.9864626,
      40.7266739
    ],
    street: 'East 6 Street',
    zipcode: '10003'
  },
  borough: 'Manhattan',
  cuisine: 'Indian',
  grades: [
    {
      date: 2014-09-15T00:00:00.000Z,
      grade: 'A',
      score: 5
    },
    {
      date: 2014-01-14T00:00:00.000Z,

```

10. Find restaurants in the borough Bronx that prepare **either American or Chinese** cuisine.

```

db.restaurants.find({
  borough: "Bronx",
  cuisine: { $in: ["American", "Chinese"] }
})

```

```

> db.restaurants.find({
  borough: "Bronx",
  cuisine: { $in: ["American", "Chinese"] }
})
< {
  _id: ObjectId('5eb3d668b31de5d588f42933'),
  address: {
    building: '2300',
    coord: [
      -73.8786113,
      40.8502883
    ],
    street: 'Southern Boulevard',
    zipcode: '10460'
  },
  borough: 'Bronx',
  cuisine: 'American',
  grades: [
    {
      date: 2014-05-28T00:00:00.000Z,
      grade: 'A',
      score: 11
    }
  ]
}

```

11. Find restaurants that are **not located** in Staten Island, Queens, Bronx, or Brooklyn.

```

db.restaurants.find({
  borough: { $nin: ["Staten Island", "Queens", "Bronx", "Brooklyn"] }
})

```

```
db.restaurants.find({
  borough: { $nin: ["Staten Island", "Queens", "Bronx", "Brooklyn"] }
})
{
  _id: ObjectId('5eb3d668b31de5d588f42937'),
  address: {
    building: '1',
    coord: [
      -73.96926909999999,
      40.7685235
    ],
    street: 'East 66 Street',
    zipcode: '10065'
  },
  borough: 'Manhattan',
  cuisine: 'American',
  grades: [
    {
      date: 2014-05-07T00:00:00.000Z,
      grade: 'A'
```

E. Sorting

12. Display all restaurants sorted by **name in ascending order**.

```
db.restaurants.find().sort({ name: 1 })
```



```
> db.restaurants.find().sort({ name: 1 })
< {
  _id: ObjectId('5eb3d669b31de5d588f48a9f'),
  address: {
    building: '308',
    coord: [
      -73.9372612,
      40.7966002
    ],
    street: 'E 116Th St',
    zipcode: '10029'
  },
  borough: 'Manhattan',
  cuisine: 'Other',
  grades: [],
  name: '',
  restaurant_id: '50018088'
}
{
  _id: ObjectId('5eb3d669b31de5d588f48bab'),
  address: {
    building: '10702',
    coord: [
      -73.8560174,
      40.7434819
    ],
    street: 'E 116Th St',
    zipcode: '10029'
  },
  borough: 'Manhattan',
  cuisine: 'Other',
  grades: [],
  name: '',
  restaurant_id: '50018088'
}
```

13. Display all restaurants sorted by **name in descending order**.

```
db.restaurants.find().sort({ name: -1 })
```

```

db.restaurants.find().sort({ name: -1 })
{
  _id: ObjectId('5eb3d669b31de5d588f46d98'),
  address: {
    building: '1',
    coord: [
      -74.073156,
      40.6457369
    ],
    street: 'Richmond Terrace',
    zipcode: '10301'
  },
  borough: 'Staten Island',
  cuisine: 'Pizza',
  grades: [
    {
      date: 2015-01-13T00:00:00.000Z,
      grade: 'Z',
      score: 18
    }
  ],
  }
}

```

F. Pattern Matching (Regex)

14. Find restaurants whose **name starts with “Wil”**. Display restaurant_id, name, borough, and cuisine.

```

db.restaurants.find(
  { name: { $regex: "^Wil" } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)

```

```

> db.restaurants.find(
  { name: { $regex: "^Wil" } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
{
  borough: 'Brooklyn',
  cuisine: 'Delicatessen',
  name: "Wilken'S Fine Food",
  restaurant_id: '40356483'
}
{
  borough: 'Bronx',
  cuisine: 'American',
  name: 'Wild Asia',
  restaurant_id: '40357217'
}

```

15. Find restaurants whose **name contains “Reg” anywhere**. Display restaurant_id, name, borough, and cuisine.

```

db.restaurants.find(
  { name: { $regex: "Reg" } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)

```

```

type: 1, ... } } }
> db.restaurants.find(
  { name: { $regex: "Reg" } },
  { restaurant_id: 1, name: 1, borough: 1, cuisine: 1, _id: 0 }
)
< {
  borough: 'Brooklyn',
  cuisine: 'American',
  name: 'Regina Caterers',
  restaurant_id: '40356649'
}
{
  borough: 'Manhattan',
  cuisine: 'Café/Coffee/Tea',
  name: 'Caffe Reggio',
  restaurant_id: '40369418'
}

```

G. Array & Embedded Document Queries

16. Find restaurants that achieved a **grade “A” with a score of 11** on the date `ISODate("2014-08-11T00:00:00Z")`.

```

db.restaurants.find(
  {
    grades: {
      $elemMatch: {
        grade: "A",
        score: 11,
        date: ISODate("2014-08-11T00:00:00Z")
      }
    }
  }
)

```

```

db.restaurants.find(
  {
    grades: {
      $elemMatch: {
        grade: "A",
        score: 11,
        date: ISODate("2014-08-11T00:00:00Z")
      }
    }
  }
)
{
  _id: ObjectId('5eb3d668b31de5d588f42a85'),
  address: {
    building: '1133',
    coord: [
      -73.958885,
      40.7745559
    ],
    street: 'Lexington Avenue',
    zipcode: '10075'
  },
  borough: 'Manhattan',
  cuisine: 'Italian',
  grades: [
    {

```

17. Find restaurants where the **second element** of the grades array contains a **grade “A” and score 9** on ISODate("2014-08-11T00:00:00Z").

```

db.restaurants.find(
  {
    "grades.1.grade": "A",

```

```

    "grades.1.score": 9,
    "grades.1.date": ISODate("2014-08-11T00:00:00Z")
  }
)

```

```

db.restaurants.find(
  {
    "grades.1.grade": "A",
    "grades.1.score": 9,
    "grades.1.date": ISODate("2014-08-11T00:00:00Z")
  }
)
{
  _id: ObjectId('5eb3d668b31de5d588f42f52'),
  address: {
    building: '26',
    coord: [
      -73.96936149999999,
      40.7658603
    ],
    street: 'East 63 Street',
    zipcode: '10065'
  },
  borough: 'Manhattan',
  cuisine: 'American',
  grades: [

```

18. Find restaurants that have **at least one grade with a score less than 5**.

```
db.restaurants.find(
```

```

{
  grades: {
    $elemMatch: {
      score: { $lt: 5 }
    }
  }
}

```

```

    }
  }
}
)

```

```

db.restaurants.find(
  {
    grades: {
      $elemMatch: {
        score: { $lt: 5 }
      }
    }
  }
)
{
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  address: {
    building: '1007',
    coord: [
      -73.856077,
      40.848447
    ],
    street: 'Morris Park Ave',
    zipcode: '10462'
  },

```

H. Advanced Array Logic

19. Find restaurants that have **both** a grade with score **2** and a grade with score **6**.

```

db.restaurants.find(
{
  $and: [
    { grades: { $elemMatch: { score: 2 } } },
    { grades: { $elemMatch: { score: 6 } } }
  ]
}
)

```

```
]
}
)
```

```
db.restaurants.find(
{
  $and: [
    { grades: { $elemMatch: { score: 2 } } },
    { grades: { $elemMatch: { score: 6 } } }
  ]
}
)
{
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  address: {
    building: '1007',
    coord: [
      -73.856077,
      40.848447
    ],
    street: 'Morris Park Ave',
    zipcode: '10462'
  },
  borough: 'Bronx',
  cuisine: 'Bakery',
  grades: [
```

20. Find restaurants that have **either** a grade with score **2 or 6**.

```
db.restaurants.find(
{
  grades: {
    $elemMatch: {
      score: { $in: [2, 6] }
    }
  }
}
```



```

    }
  }
)
db.restaurants.find(
  {
    grades: {
      $elemMatch: {
        score: { $in: [2, 6] }
      }
    }
  }
)
{
  _id: ObjectId('5eb3d668b31de5d588f4292e'),
  address: {
    building: '1007',
    coord: [
      -73.856077,
      40.848447
    ],
    street: 'Morris Park Ave',
    zipcode: '10462'
  },
  borough: 'Bronx',
  cuisine: 'Bakery',
  grades: [

```

21. Find restaurants where **all grades** have a score **greater than 5**.

```

db.restaurants.find(
{
  grades: {
    $not: {
      $elemMatch: { score: { $lte: 5 } }
    }
  }
}

```

```
}  
}  
)
```

```
db.restaurants.find(  
  {  
    grades: {  
      $not: {  
        $elemMatch: { score: { $lte: 5 } }  
      }  
    }  
  }  
)  
{  
  _id: ObjectId('5eb3d668b31de5d588f4292b'),  
  address: {  
    building: '7114',  
    coord: [  
      -73.9068506,  
      40.6199034  
    ],  
    street: 'Avenue U',  
    zipcode: '11234'  
  },  
  borough: 'Brooklyn',  
  cuisine: 'Delicatessen',  
  grades: [  

```

Submission Requirements

- MongoDB query code for each question
- Screenshots showing successful execution and results
- Submit as a single PDF or Word document